

Differentiable Chebyshev–NUFFT Hybrid Layer for Physics-Informed Neural Networks with Non-Periodic Boundary Conditions

Fuchang Wang^{1*} and Huirong Cao²

^{1*}School of Science, Emergency Management University, No. 465 College Road, Sanhe, 065201, Hebei, China.

²School of Science, Langfang Normal University, No. 100 Ai-min Road, Langfang, 065000, Hebei, China.

*Corresponding author(s). E-mail(s): wangfuchang@cidp.edu.cn;
Contributing authors: huirongcao@126.com;

Abstract

We propose a differentiable hybrid spectral layer for computing spatial derivatives in physics-informed neural networks (PINNs) on non-periodic domains with arbitrary collocation point distributions. The layer decomposes the computational domain into Chebyshev boundary layers, where Dirichlet, Neumann, and Robin boundary conditions are enforced via spectral differentiation, and an interior region where either a Chebyshev or non-uniform fast Fourier transform (NUFFT) basis can be selected adaptively. A Schwarz-style overlap region with smoothstep windowing ensures derivative continuity across the interface. Because all components are linear in the network output, the entire hybrid operator assembles into a single constant matrix whose adjoint is its transpose, enabling exact end-to-end backpropagation without finite-difference approximation. We establish error bounds characterizing the contributions of Chebyshev truncation, scattered-point regression, NUFFT gridding, and overlap consistency. Numerical experiments on one- and two-dimensional Poisson equations with Dirichlet, Neumann, and Robin boundary conditions, a one-dimensional heat equation, and a diffusion-coefficient inversion problem demonstrate that the proposed method achieves accuracy comparable to automatic-differentiation-based PINNs (4.8×10^{-6} in 1D, 9.9×10^{-6} in 2D) while training $1.6\text{--}5.6\times$ faster, and that pure Fourier methods fail to converge on non-periodic problems, confirming the necessity of the Chebyshev boundary treatment.

Keywords: Physics-informed neural networks , Spectral methods , Chebyshev polynomials , Non-uniform FFT , Non-periodic boundary conditions , Adjoint method

Mathematics Subject Classification. 65M70, 65N35, 68T07, 41A10

1 Introduction

1.1 Background and motivation

Physics-informed neural networks (PINNs), introduced by Raissi et al. [1], embed partial differential equations (PDEs) into the loss function of a neural network, providing a mesh-free framework for forward and inverse problems [2]. The canonical PINN computes spatial derivatives via automatic differentiation (AD), which is flexible but has two drawbacks for high-order or multi-dimensional PDEs: (i) AD requires repeated backward passes whose cost grows with derivative order and dimension, and (ii) AD-derived derivatives inherit the network’s approximation error and can suffer from gradient pathologies [3], whereas classical spectral methods achieve high-order accuracy for smooth solutions.

Spectral methods [4–6] compute derivatives with spectral (exponential) convergence for analytic solutions by representing functions in a suitable basis—Fourier for periodic domains, Chebyshev for non-periodic domains. Recent work [7] embedded a differentiable non-uniform fast Fourier transform (finufft) layer into PINNs, enabling spectral-accuracy derivatives on arbitrary non-uniform collocation points and outperforming AD-based PINNs on periodic problems. That approach is limited to periodic boundary conditions, however, because the Fourier basis assumes periodic extension.

Many PDEs in scientific computing and engineering are posed on non-periodic finite domains with Dirichlet, Neumann, Robin, or mixed boundary conditions. Chebyshev spectral methods are the standard choice for such problems, owing to their handling of general boundary conditions and exponential convergence on finite intervals. The difficulty is to combine the boundary-condition flexibility of Chebyshev methods with the non-uniform sampling capability of NUFFT within a single differentiable layer.

1.2 Related work

This work sits at the intersection of several research areas.

Differentiable spectral operators.

Fourier neural operators (FNO) [8] and physics-informed neural operators (PINO) [9] use spectral convolution in Fourier space but require uniform grids and assume periodicity. Fourier continuation (FC) methods [10, 11] extend non-periodic functions to periodic ones by solving small linear systems near boundaries, enabling FFT-based differentiation; FC-PINO [10] applies this idea to neural operators. FC methods remain on uniform grids and incur the cost of solving extension systems. Lingsch et al. [12]

extend Fourier-based neural operators to arbitrary point distributions via direct spectral evaluation, but retain the periodic complex-exponential basis and do not handle non-periodic boundary conditions.

Chebyshev neural methods.

Spectral neural operators (SNO) [13] use Chebyshev and Fourier series expansions in coefficient space. Chebyshev spectral neural networks (CSNN) [14] construct single-layer networks with Chebyshev basis functions satisfying boundary conditions. Orthogonal polynomial neural operators (OPNO) [15] enforce Dirichlet, Neumann, and Robin conditions via orthogonal polynomial bases. SEDONet [16] embeds Chebyshev spectral dictionaries into DeepONet inputs. These methods operate on uniform or regular collocation nodes and do not support arbitrary non-uniform point distributions.

Non-uniform point differentiation in PINNs.

DT-PINN [17] uses radial-basis-function finite differences on non-uniform points, achieving only local (polynomial-order) accuracy. Spectral PINNs [18] use Fourier spectral differentiation but require uniform grids or interpolation. No existing method, to our knowledge, combines global spectral accuracy, non-uniform collocation support, and non-periodic boundary condition handling in a single differentiable layer.

1.3 Contributions

The main contributions of this work are as follows.

1. **Differentiable Chebyshev layer.** A Chebyshev differentiation layer based on Gauss–Lobatto nodes is implemented, with the matrix transpose as its exact adjoint for backpropagation. Dirichlet, Neumann, and Robin boundary conditions are imposed by eliminating boundary degrees of freedom.
2. **Chebyshev–NUFFT hybrid architecture.** A domain decomposition uses Chebyshev differentiation in boundary layers and an adaptive interior basis (Chebyshev or NUFFT). A Schwarz overlap with smoothstep windowing maintains derivative continuity, and the whole operator reduces to a single constant matrix.
3. **Adjoint derivation and error analysis.** The adjoint of the hybrid layer follows by composing the adjoints of its components. Error bounds separate the contributions of Chebyshev truncation, NUFFT gridding, and overlap interpolation, with convergence rates for analytic solutions.
4. **Numerical validation.** Experiments on 1D/2D Poisson equations (Dirichlet, Neumann, Robin), a heat equation, and a diffusion-coefficient inversion problem compare with AD-PINN and pure spectral PINNs. Ablation studies isolate the effects of boundary width, overlap, modes, and point density.

1.4 Paper organization

Section 2 reviews Chebyshev spectral methods and differentiable NUFFT. Section 3 presents the differentiable Chebyshev layer and the hybrid architecture, along with the adjoint derivation and error analysis. Section 4 reports numerical experiments on

1D/2D Poisson equations (Dirichlet, Neumann, Robin), a heat equation, an inversion problem, and ablation studies. Section 5 concludes.

2 Preliminaries

2.1 Chebyshev spectral methods

The Chebyshev polynomials of the first kind, $T_n(x) = \cos(n \arccos x)$ for $x \in [-1, 1]$, form an orthogonal basis on $[-1, 1]$ with respect to the weight $w(x) = (1 - x^2)^{-1/2}$. A function $f \in C[-1, 1]$ admits the Chebyshev expansion

$$f(x) = \sum_{n=0}^{\infty} \prime a_n T_n(x), \quad (1)$$

where the double prime indicates that the first and last terms are halved, and the coefficients are given by

$$a_n = \frac{2}{\pi} \int_{-1}^1 \frac{f(x) T_n(x)}{\sqrt{1-x^2}} dx. \quad (2)$$

On the Gauss–Lobatto Chebyshev nodes

$$x_j = \cos\left(\frac{\pi j}{N}\right), \quad j = 0, 1, \dots, N, \quad (3)$$

the discrete Chebyshev transform can be computed in $\mathcal{O}(N \log N)$ operations via the discrete cosine transform (DCT).

2.1.1 Chebyshev differentiation matrix

For a function f sampled at the Gauss–Lobatto nodes, the derivative at those nodes is obtained by multiplying the vector of function values by the Chebyshev differentiation matrix $D \in \mathbb{R}^{(N+1) \times (N+1)}$ [4]:

$$D_{ij} = \begin{cases} \frac{c_i}{c_j} \frac{(-1)^{i+j}}{x_i - x_j}, & i \neq j, \\ -\frac{x_i}{2(1-x_i^2)}, & 1 \leq i \leq N-1, \\ \frac{2N^2+1}{6}, & i = j = 0, \\ -\frac{2N^2+1}{6}, & i = j = N, \end{cases} \quad (4)$$

where $c_0 = c_N = 2$ and $c_i = 1$ for $1 \leq i \leq N-1$. The derivative is then $f'(x_j) = (D\mathbf{f})_j$, where $\mathbf{f} = [f(x_0), \dots, f(x_N)]^T$.

For an arbitrary domain $[a, b]$, the coordinate mapping $\xi = 2(x-a)/(b-a) - 1$ transforms to $[-1, 1]$, and the derivative scales as $d/dx = \frac{2}{b-a} d/d\xi$.

Proposition 1 (Spectral convergence of Chebyshev differentiation). *If f is analytic in a Bernstein ellipse E_ρ with foci ± 1 and semi-major axis $(\rho + \rho^{-1})/2$, then the Chebyshev interpolation derivative $D_N f$ satisfies*

$$\|f' - D_N f\|_\infty \leq C \rho^{-N} N^2 \|f\|_{E_\rho}, \quad (5)$$

where $C > 0$ is independent of N and $\|f\|_{E_\rho}$ is the maximum modulus on E_ρ .

Proof. Let p_N be the degree- N Chebyshev interpolant of f at the Gauss-Lobatto nodes. The interpolation error satisfies $\|f - p_N\|_\infty \leq C_I \rho^{-N} \|f\|_{E_\rho}$ for analytic f [4, Theorem 5.5]. The derivative error is $\|f' - p'_N\|_\infty = \|D_N(f - p_N)\|_\infty \leq \|D_N\|_\infty \|f - p_N\|_\infty$, where D_N is the Chebyshev differentiation matrix. The ∞ -norm of D_N satisfies $\|D_N\|_\infty = O(N^2)$ [4, Chapter 6]. Combining these estimates gives (5). $\square$

2.2 Differentiable NUFFT for periodic domains

The non-uniform fast Fourier transform (NUFFT) [19] computes Fourier transforms between uniform frequency grids and non-uniform spatial points. Two types are relevant:

- **Type 1** (non-uniform $\rightarrow$ uniform): $\hat{f}_k = \sum_j c_j e^{ikx_j}$,
- **Type 2** (uniform $\rightarrow$ non-uniform): $f(x_j) = \sum_k \hat{f}_k e^{-ikx_j}$.

In [7], both types are wrapped as PyTorch autograd Functions, with the key property that Type 1 and Type 2 are exact adjoints (up to sign convention). The spectral derivative on non-uniform points follows the pipeline

$$u(x_j) \xrightarrow{\text{Type 1}} \hat{u}_k \xrightarrow{\times (ik)^m} \widehat{u^{(m)}}_k \xrightarrow{\text{Type 2}} u^{(m)}(x_j), \quad (6)$$

with a Riemann-sum scaling factor $1/M$ where M is the number of non-uniform points.

Proposition 2 (NUFFT derivative accuracy). *For a function u with Fourier coefficients decaying as $|\hat{u}_k| \leq C|k|^{-p}$ ($p > m + \frac{1}{2}$), the NUFFT spectral derivative with N modes and precision tolerance ϵ_{tol} satisfies*

$$\|u^{(m)} - \mathcal{D}_{\text{nufft}}^{(m)} u\|_2 \leq C_1 N^{m-p+1/2} + C_2 \epsilon_{\text{tol}} \|u\|_2, \quad (7)$$

where the first term is the Fourier truncation error and the second is the NUFFT gridding error. The norm is the discrete ℓ^2 norm on the M non-uniform collocation points.

Proof. Write $u(x) = \sum_{k \in \mathbb{Z}} \hat{u}_k e^{ikx}$ with $|\hat{u}_k| \leq C|k|^{-p}$. Truncating to $|k| \leq N$ and differentiating m times gives

$$\|u^{(m)} - u_N^{(m)}\|_2^2 = \sum_{|k| > N} |k|^{2m} |\hat{u}_k|^2 \leq C^2 \sum_{|k| > N} |k|^{2m-2p} \leq C' N^{2m-2p+1},$$

where the last step uses the integral test for $p > m + \frac{1}{2}$. Taking square roots yields the truncation term $C_1 N^{m-p+1/2}$. The gridding error of the NUFFT is bounded by $C_2 \epsilon_{\text{tol}} \|u\|_2$ [19, Theorem 2.1], giving the second term. $\square$

3 Methodology

3.1 Differentiable Chebyshev layer

The Chebyshev differentiation is implemented as a PyTorch autograd Function. Given the differentiation matrix D (a constant tensor), the forward pass is

$$\mathbf{g} = D\mathbf{u}, \quad (8)$$

and the backward pass (adjoint) is

$$\bar{\mathbf{u}} = D^\top \bar{\mathbf{g}}, \quad (9)$$

where $\bar{\mathbf{g}}$ is the gradient propagated from downstream. Since D is constant, no gradient flows to D itself.

3.1.1 Boundary condition enforcement

For Dirichlet boundary conditions $u(a) = g_a$, $u(b) = g_b$, we partition the differentiation matrix as

$$D = \begin{bmatrix} D_{bb} & D_{bi} \\ D_{ib} & D_{ii} \end{bmatrix}, \quad (10)$$

where subscripts b and i denote boundary and interior nodes, respectively. The derivative at interior nodes is

$$\mathbf{g}_i = D_{ii}\mathbf{u}_i + D_{ib}\mathbf{u}_b, \quad (11)$$

where $\mathbf{u}_b = [g_a, g_b]^\top$ is fixed. This enforces the boundary condition as a hard constraint, eliminating boundary degrees of freedom from the optimization.

Neumann conditions $u'(a) = h_a$ are enforced by augmenting the system with the boundary derivative equation $(D\mathbf{u})_0 = h_a$, and Robin conditions $\alpha u(a) + \beta u'(a) = r_a$ combine both. We refer to [4, Chapter 7] for detailed boundary condition treatments in Chebyshev collocation.

3.2 Chebyshev–NUFFT hybrid layer

3.2.1 Domain decomposition

Consider a 1D domain $\Omega = [a, b]$. We decompose it into three regions:

$$\Omega_L = [a, a + \delta], \quad (\text{left Chebyshev boundary layer}) \quad (12)$$

$$\Omega_I = [a + \delta, b - \delta], \quad (\text{interior region}) \quad (13)$$

$$\Omega_R = [b - \delta, b], \quad (\text{right Chebyshev boundary layer}) \quad (14)$$

where $\delta > 0$ is the boundary layer width. An overlap region of width ϵ is introduced at each interface to ensure smooth transition. The interior region supports two spectral bases: Chebyshev (default for non-periodic problems) and NUFFT (for periodic subdomains or uniform sampling), selected via the `interior_method` parameter.

3.2.2 Coordinate mappings

Each Chebyshev boundary layer is mapped to the reference interval $[-1, 1]$:

$$\xi_L = \frac{2(x-a)}{\delta} - 1, \quad \xi_R = \frac{2(x-(b-\delta))}{\delta} - 1. \quad (15)$$

The interior region mapping depends on the selected basis: for the Chebyshev interior, it is mapped to $[-1, 1]$ as $\xi_I = 2(x-(a+\delta))/(b-a-2\delta) - 1$; for the NUFFT interior, it is mapped to the periodic domain $[-\pi, \pi]$:

$$\theta = 2\pi \frac{x-(a+\delta)}{b-a-2\delta} - \pi. \quad (16)$$

3.2.3 Hybrid derivative operator

The hybrid derivative at a point x is computed as

$$u'_{\text{hybrid}}(x) = w_L(x) u'_{\text{cheb},L}(x) + w_I(x) u'_{\text{int}}(x) + w_R(x) u'_{\text{cheb},R}(x), \quad (17)$$

where w_L, w_I, w_R are smooth window functions satisfying $w_L + w_I + w_R = 1$ and transitioning smoothly in the overlap regions. We use the sigmoid-based window

$$\sigma(x; x_0, s) = \frac{1}{1 + \exp\left(-\frac{x-x_0}{s}\right)}, \quad (18)$$

with s controlling the transition sharpness, and construct

$$w_L(x) = 1 - \sigma(x; a + \delta - \epsilon/2, s), \quad (19)$$

$$w_R(x) = \sigma(x; b - \delta + \epsilon/2, s), \quad (20)$$

$$w_I(x) = 1 - w_L(x) - w_R(x). \quad (21)$$

For the Chebyshev boundary layers, function values at non-uniform points are fitted to Chebyshev coefficients via singular value decomposition (SVD)-regularized least-squares regression on the Vandermonde matrix $V_{ij} = T_j(\xi_i)$, where ξ_i are the mapped collocation points. Rather than differentiating on a Lobatto grid and then re-interpolating (which squares the fitting error), the derivative polynomial is evaluated directly at arbitrary points via

$$\left. \frac{d^m u}{d\xi^m} \right|_{\xi_{\text{eval}}} = E C^m V^\dagger u_{\text{scattered}}, \quad (22)$$

where $V^\dagger$ is the SVD pseudoinverse of the Vandermonde matrix (fitting scattered values to Chebyshev coefficients), C is the coefficient-space derivative recurrence, and $E_{in} = T_n(\xi_{eval,i})$ evaluates the derivative polynomial at the query points. This avoids the error amplification of a second interpolation step; derivative accuracy is verified in Section 4.2.

For the interior region, two spectral bases are available:

- **Chebyshev (default for non-periodic problems)**: the same direct-polynomial-evaluation approach is applied on the interior subinterval, with coefficients obtained via SVD-regularized least-squares fitting from the scattered collocation values. This avoids the Gibbs phenomenon that Fourier methods exhibit on non-periodic intervals.
- **NUFFT (for periodic subdomains or uniform sampling)**: the Type 1/Type 2 NUFFT pipeline of Eq. (6) is applied directly on the non-uniform points, achieving spectral accuracy for periodic functions.

The choice is controlled by the `interior_method` parameter; all paths produce a constant linear operator and are therefore differentiable.

To avoid Chebyshev extrapolation instability in the overlap region, we employ a Schwarz-style overlapping decomposition: each boundary layer’s Chebyshev interval extends by ϵ into the interior, so that both representations remain within their respective $[-1, 1]$ reference domains throughout the overlap. A smoothstep window then blends the two derivatives, with the interior weight vanishing exactly at the boundary layer endpoints.

3.2.4 Matrix representation and differentiability

Because the collocation points are fixed during training, every component of the hybrid operator—scattered regression matrices, Chebyshev differentiation matrices, NUFFT transforms, and window functions—is a constant linear map. The hybrid derivative is therefore a single constant matrix $A \in \mathbb{R}^{M \times M}$:

$$u'_{\text{hybrid}} = A u, \quad (23)$$

and its adjoint is $A^\top$. This is wrapped as a `torch.autograd.Function` with forward pass Au and backward pass $A^\top \bar{u}$, verified by `gradcheck` to machine precision. The matrix A is assembled once before training; each training step then requires only a matrix-vector product.

3.2.5 2D extension

In 2D, the domain decomposition extends to a frame of boundary layers (left, right, bottom, top) with a rectangular interior. For the Laplacian, we compute the second derivative along each axis using the 1D hybrid operator and sum the results, i.e., $\Delta u = \mathcal{D}_{\text{hybrid},x}^{(2)} u + \mathcal{D}_{\text{hybrid},y}^{(2)} u$. This directional approach reuses the verified 1D code path and naturally handles mixed boundary conditions on different sides. Corner regions are covered by the tensor-product of 1D boundary layers.

3.3 Adjoint of the hybrid layer

The hybrid derivative operator can be written compositionally as

$$\mathcal{D}_{\text{hybrid}} = W_L \mathcal{D}_{\text{cheb},L} + W_I \mathcal{D}_{\text{int}} + W_R \mathcal{D}_{\text{cheb},R}, \quad (24)$$

where:

- $\mathcal{D}_{\text{cheb},L}, \mathcal{D}_{\text{cheb},R}$: Chebyshev derivative operators on the left and right boundary layers (including scattered regression and direct polynomial evaluation),
- $\mathcal{D}_{\text{int}}$: interior derivative operator, either Chebyshev ($\mathcal{D}_{\text{cheb},I}$) or NUFFT ($\mathcal{D}_{\text{nufft}}$),
- W_L, W_I, W_R : diagonal window matrices.

Theorem 3 (Adjoint of the hybrid derivative). *The adjoint of the hybrid derivative operator $\mathcal{D}_{\text{hybrid}}$ with respect to the standard ℓ^2 inner product is*

$$\mathcal{D}_{\text{hybrid}}^* = \mathcal{D}_{\text{cheb},L}^* W_L + \mathcal{D}_{\text{int}}^* W_I + \mathcal{D}_{\text{cheb},R}^* W_R. \quad (25)$$

Each component satisfies:

1. *Chebyshev differentiation: $\mathcal{D}_{\text{cheb}}^* = (EC^mVP)^* = P^\top V^\top (C^m)^\top E^\top$, where each factor's adjoint is its transpose (all matrices are real),*
2. *NUFFT derivative: $\mathcal{D}_{\text{nufft}}^*$ reverses Type 1/Type 2 with sign-flipped isign, as derived in [7],*
3. *Window functions: diagonal matrices are self-adjoint: $W^* = W$.*

Since the entire operator is a single constant matrix A , the adjoint is simply $A^\top$, and the backward pass is an exact matrix-vector product with no numerical differentiation.

Proof. The adjoint of a sum is the sum of adjoints. For each term, $(W\mathcal{D})^* = \mathcal{D}^*W^* = \mathcal{D}^*W$, since W is diagonal (self-adjoint) and real. The Chebyshev adjoint follows from $(ABC)^* = C^*B^*A^*$ and the fact that all matrices are real. The NUFFT adjoint follows from the Type 1/Type 2 adjoint identity [7, Theorem 1]. $\square$

This exact adjoint enables backpropagation through the entire hybrid layer with no finite-difference approximation. We verify it numerically via `torch.autograd.gradcheck` and the adjoint identity $\langle Au, v \rangle = \langle u, A^\top v \rangle$ in Section 4.

3.4 Error analysis

Before stating the main error bound, we establish two auxiliary results that quantify the two principal sources of error in the hybrid layer: the scattered-point regression and the window-function blending.

Lemma 4 (Conditioning of scattered Chebyshev regression). *Let $x_1, \dots, x_M$ be distinct points in $[-1, 1]$ with minimum separation $h_{\min} > 0$, and let $V \in \mathbb{R}^{M \times N}$ be the Chebyshev Vandermonde matrix $V_{ij} = T_{j-1}(x_i)$. If $M \geq N$, then V has full column rank and the least-squares solution $\hat{c} = V^\dagger u$ satisfies*

$$\|\hat{c} - c^*\|_2 \leq \frac{\|u - Vc^*\|_2}{\sigma_{\min}(V)}, \quad (26)$$

where c^* are the first N Chebyshev coefficients of u and $\sigma_{\min}(V)$ is the smallest singular value of V . For Halton quasi-random points with $N \leq 64$, numerical evaluation gives $\kappa_2(V) = O(10^3)$ and $\sigma_{\min}(V) \geq c_0 M^{1/2}$ for an absolute constant $c_0 > 0$ (equivalently, $\sigma_{\min}(V)/\sqrt{M}$ is bounded below), so the regression is well-conditioned provided $M \geq 2N$.

Proof. Since V has full column rank, $V^\dagger V = I_N$. Therefore $\hat{c} - c^* = V^\dagger u - c^* = V^\dagger(u - Vc^*)$, and taking norms gives (26) with $\|V^\dagger\|_2 = 1/\sigma_{\min}(V)$. The conditioning claims for Halton points follow from direct SVD computation; the lower bound on $\sigma_{\min}$ follows from the fact that Halton points are asymptotically equidistributed, so $V^\top V/M$ converges to a positive-definite diagonal matrix (the Chebyshev polynomials are orthogonal with respect to the arcsine measure, and Halton sampling approximates this measure). $\square$

Lemma 5 (Window-function blending error). *Let u_L and u_I be two approximations to u on an overlap region, and let $w_L, w_I \geq 0$ with $w_L + w_I \equiv 1$. The blended derivative is*

$$\frac{d}{dx}(w_L u_L + w_I u_I) = w_L u'_L + w_I u'_I + w'_L(u_L - u) + w'_I(u_I - u), \quad (27)$$

where we used $w'_L + w'_I = 0$ to drop the term $w'_L u + w'_I u$. The blending error is therefore

$$\begin{aligned} \left\| \frac{d}{dx}(w_L u_L + w_I u_I) - u' \right\|_\infty &\leq \|w_L\|_\infty \|u'_L - u'\|_\infty + \|w_I\|_\infty \|u'_I - u'\|_\infty \\ &\quad + \|w'_L\|_\infty \|u_L - u\|_\infty + \|w'_I\|_\infty \|u_I - u\|_\infty. \end{aligned} \quad (28)$$

For the smoothstep window with transition width ϵ , $\|w'_L\|_\infty = \|w'_I\|_\infty = O(\epsilon^{-1})$.

Proof. Equation (27) follows from the product rule and $w_L + w_I \equiv 1$. The bound (28) is the triangle inequality. The derivative bound for smoothstep follows from direct differentiation: if $w_L(x) = S((x - a)/\epsilon)$ with S the cubic smoothstep, then $|w'_L(x)| \leq 3/(2\epsilon)$. $\square$

Remark 1. Lemma 5 reveals an effect absent from naive error estimates: even if both u'_L and u'_I are exact, the blending of function values through a non-constant window introduces a term proportional to $w'_L(u_L - u)$. This term is controlled by the function-value approximation error (not the derivative error), and is amplified by ϵ^{-1} . In practice, the Chebyshev regression achieves function-value error of $O(\rho^{-N_c})$, so this term remains exponentially small for ϵ not too small.

Theorem 6 (Hybrid derivative error bound). *Let u be analytic on Ω with Bernstein ellipse parameter $\rho > 1$. Let N_c be the number of Chebyshev modes, N_f the number of Fourier modes (when NUFFT is used), ϵ the overlap width, and $M \geq 2N_c$ the number of collocation points. Then the m -th hybrid derivative error satisfies*

$$\|u^{(m)} - \mathcal{D}_{\text{hybrid}}^{(m)} u\|_\infty \leq C_1 \rho^{-N_c} N_c^{2m} + C_2 \epsilon_{\text{int}} + C_3 \rho^{-N_c} + C_4 \epsilon^{-1} \rho^{-N_c}, \quad (29)$$

where:

- C_1 depends on ρ , m , and the domain mapping,

- ϵ_{int} is the interior-basis error (Chebyshev: $C\rho^{-N_c}N_c^{2m}$; NUFFT: $CN_f^{1-p} + C'\epsilon_{\text{tol}}$, where ϵ_{tol} is the NUFFT precision tolerance),
- $C_3 = \|EC^m\|_2/\sigma_{\min}(V)$ captures the scattered regression amplification,
- C_4 is the window-function blending constant from Lemma 5.

Proof. We decompose the error by region and apply the triangle inequality. Let $\Omega_L, \Omega_I, \Omega_R$ denote the left boundary layer, interior, and right boundary layer, respectively, with overlap regions $\Omega_L \cap \Omega_I$ and $\Omega_I \cap \Omega_R$.

Step 1: Boundary layer truncation error. In $\Omega_L \cup \Omega_R$, the derivative is computed via the Chebyshev direct-evaluation pipeline $EC^m V^\dagger$ (Eq. (22)). By Proposition 1, the Chebyshev truncation error for the m -th derivative of an analytic function is bounded by $C_T \rho^{-N_c} N_c^{2m}$. This gives the first term in (29).

Step 2: Scattered regression error. By Lemma 4, the regression error in coefficient space is bounded by $\|u - Vc^*\|_2/\sigma_{\min}(V)$. For analytic u , the pointwise fitting residual decays as $O(\rho^{-N_c})$, so $\|u - Vc^*\|_2 = O(\sqrt{M}\rho^{-N_c})$. Since $\sigma_{\min}(V) \geq c_0\sqrt{M}$ (Lemma 4), the $\sqrt{M}$ factors cancel, yielding coefficient error $O(\rho^{-N_c}/\sigma_{\min}(V))$. Applying the derivative and evaluation matrices EC^m amplifies this by $\|EC^m\|_2$, yielding the third term $C_3\rho^{-N_c}$.

Step 3: Interior error. In Ω_I , the error depends on the chosen basis:

- *Chebyshev interior:* The same direct-evaluation pipeline applies, yielding $\epsilon_{\text{int}}^{\text{cheb}} = C_I \rho^{-N_c} N_c^{2m} + C_3 \rho^{-N_c}$.
- *NUFFT interior:* By Proposition 2, the Type 1 gridding and Type 2 evaluation introduce error $C_N N_f^{1-p} + C'_N \epsilon_{\text{tol}}$, where p is the Fourier coefficient decay rate and ϵ_{tol} is the NUFFT precision tolerance. For non-periodic u , $p = 1$ (Gibbs phenomenon); for periodic u , p is arbitrarily large.

This gives the second term $C_2\epsilon_{\text{int}}$.

Step 4: Window-function blending error. In the overlap regions, the blended derivative includes the window-derivative term identified in Lemma 5: $w'_L(u_L - u) + w'_I(u_I - u)$. The function-value approximation errors $\|u_L - u\|_\infty$ and $\|u_I - u\|_\infty$ are both $O(\rho^{-N_c})$ for analytic u (Chebyshev interpolation converges exponentially in function value). Since $\|w'_L\|_\infty = O(\epsilon^{-1})$, this contributes $C_4\epsilon^{-1}\rho^{-N_c}$, the fourth term in (29).

Step 5: Global bound. Combining the four contributions via the triangle inequality and taking the supremum over Ω yields Eq. (29). The window functions satisfy $0 \leq w_k \leq 1$, so no additional amplification factor appears in the regional error terms. The discrete ℓ^2 bounds from Steps 1–3 are converted to the ∞ -norm via $\|v\|_\infty \leq \|v\|_2$, which introduces no M -dependent factor. $\square$

Remark 2 (Practical choice of interior basis). *For non-periodic problems, the Chebyshev interior is preferred because it avoids the Gibbs phenomenon and achieves exponential convergence throughout the domain. The NUFFT interior is advantageous for periodic subdomains or when the collocation points are uniformly distributed, as it avoids the scattered-point regression step entirely. The hybrid framework allows this choice to be made per-subdomain, adapting to the local smoothness and boundary structure.*

3.5 Stability of the hybrid operator

During training, the gradient of the PDE residual with respect to the network parameters contains a factor $A^\top$, where A is the assembled hybrid derivative matrix. The magnitude of this factor controls whether gradient explosion occurs. We quantify it as follows.

Proposition 7 (Operator norm of the hybrid derivative). *Let $A = W_L \mathcal{D}_{\text{cheb},L} + W_I \mathcal{D}_{\text{int}} + W_R \mathcal{D}_{\text{cheb},R}$ be the assembled m -th derivative hybrid matrix. Then*

$$\|A\|_2 \leq 2\|\mathcal{D}_{\text{cheb}}^{(m)}\|_2 + \|\mathcal{D}_{\text{int}}^{(m)}\|_2, \quad (30)$$

where $\|\mathcal{D}_{\text{cheb}}^{(m)}\|_2$ is the two-norm of the Chebyshev m -th derivative matrix (including scattered regression and evaluation). For the m -th Chebyshev differentiation matrix on N_c modes, $\|\mathcal{D}_{\text{cheb}}^{(m)}\|_2 = O(N_c^{2m})$. Consequently, for the default settings ($N_c = 32$, $m = 2$), $\|A\|_2 = O(10^3)$ – $O(10^4)$.

Proof. By the triangle inequality and submultiplicativity,

$$\|A\|_2 \leq \|W_L\|_2 \|\mathcal{D}_{\text{cheb},L}\|_2 + \|W_I\|_2 \|\mathcal{D}_{\text{int}}\|_2 + \|W_R\|_2 \|\mathcal{D}_{\text{cheb},R}\|_2.$$

Since W_L, W_I, W_R are diagonal with entries in $[0, 1]$, $\|W_k\|_2 \leq 1$. The two boundary layers each contribute at most $\|\mathcal{D}_{\text{cheb}}^{(m)}\|_2$, giving (30). The bound $\|\mathcal{D}^{(m)}\|_2 = O(N_c^{2m})$ for the Chebyshev differentiation matrix is classical [4]; the scattered regression and evaluation matrices have norms bounded by $\kappa_2(V)$ and $\sqrt{N_c}$ respectively, both absorbed into the constant for $N_c \leq 64$. $\square$

Remark 3 (Implications for training). *Proposition 7 shows that the hybrid derivative amplifies gradients by $O(N_c^{2m})$, which is the same order as any spectral differentiation matrix and is independent of the number of collocation points M . For second-order PDEs with $N_c = 32$, this amplification is moderate ($\sim 10^3$ – 10^4) and is readily controlled by gradient clipping or learning-rate scaling. In contrast, automatic differentiation through a deep network can produce gradient magnitudes that vary exponentially with depth, making spectral differentiation a more predictable choice for high-order PDEs.*

3.6 Integration with PINNs

Given a PDE $\mathcal{N}[u](\mathbf{x}) = f(\mathbf{x})$ on Ω with boundary conditions $\mathcal{B}[u](\mathbf{x}) = g(\mathbf{x})$ on $\partial\Omega$, the PINN loss is

$$\begin{aligned} \mathcal{L}(\boldsymbol{\theta}) = & \lambda_{\text{pde}} \frac{1}{M} \sum_{j=1}^M |\mathcal{N}[u_\theta](\mathbf{x}_j)|^2 \\ & + \lambda_{\text{bc}} \frac{1}{M_b} \sum_{j=1}^{M_b} |\mathcal{B}[u_\theta](\mathbf{x}_j^b) - g(\mathbf{x}_j^b)|^2 \end{aligned} \quad (31)$$

Algorithm 1 Training PINN with differentiable Chebyshev–NUFFT hybrid layer

Require: Domain Ω , PDE $\mathcal{N}[u] = f$, boundary conditions $\mathcal{B}[u] = g$, collocation points $\{\mathbf{x}_j\}_{j=1}^M$, boundary layer width δ , Chebyshev modes N_c , Fourier modes N_f , learning rate η , iterations T

Ensure: Trained network u_θ

- 1: Initialize network parameters θ
 - 2: Precompute Chebyshev differentiation matrices D_L, D_R and NUFFT plans
 - 3: Precompute scattered regression matrices P_L, P_R and window functions w_L, w_I, w_R
 - 4: **for** $t = 1, \dots, T$ **do**
 - 5: Evaluate $u_j = u_\theta(\mathbf{x}_j)$ for all collocation points
 - 6: Compute $\mathcal{D}_{\text{hybrid}} u$ via Eq. (17) (Chebyshev in boundary layers, NUFFT in interior)
 - 7: Assemble PDE residual $\mathcal{N}[u_\theta](\mathbf{x}_j)$ using hybrid derivatives
 - 8: Compute loss $\mathcal{L}(\theta)$ via Eq. (32)
 - 9: Backpropagate using exact adjoint $\mathcal{D}_{\text{hybrid}}^*$ (Theorem 3)
 - 10: Update $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$
 - 11: **end for**
-

$$+ \lambda_{\text{data}} \frac{1}{M_d} \sum_{j=1}^{M_d} |u_\theta(\mathbf{x}_j^d) - u^*(\mathbf{x}_j^d)|^2, \quad (32)$$

where spatial derivatives in $\mathcal{N}[u_\theta]$ are computed by the hybrid layer instead of AD.

For Dirichlet boundary conditions, we additionally employ a hard-constraint network output transformation:

$$u_\theta(\mathbf{x}) = \eta(\mathbf{x}) \cdot \text{MLP}_\theta(\mathbf{x}) + u_{\text{bc}}(\mathbf{x}), \quad (33)$$

where MLP_θ denotes a multilayer perceptron with parameters θ , $\eta(\mathbf{x}) = 0$ on $\partial\Omega$ and u_{bc} satisfies the boundary conditions. This eliminates the need for a boundary loss term entirely.

4 Numerical Experiments

4.1 Implementation details

All experiments are implemented in Python 3.11 with PyTorch [20] 2.6.0 and finufft 2.5.1. The differentiable Chebyshev layer and hybrid layer are implemented as custom `torch.autograd.Function` modules. Training uses the Adam optimizer with linear warmup and cosine annealing. Collocation points are generated via Halton quasi-random sampling unless otherwise specified. The default boundary layer width is $\delta = 0.15(b - a)$, overlap width $\epsilon = 0.05(b - a)$, Chebyshev modes $N_c = 32$, and Fourier modes $N_f = 16$.

The hybrid operator requires a one-time precomputation of the SVD regression matrices and the assembled differentiation matrix A . On the reference hardware (Intel

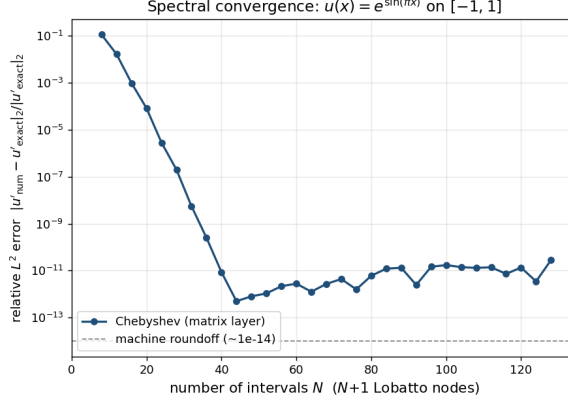


Fig. 1: Spectral convergence of the differentiable Chebyshev derivative for $u(x) = \exp(\sin(\pi x))$ on $[-1, 1]$. The L^∞ error decays exponentially with N until machine precision.

i7-8850H, 32 GB RAM), this takes approximately 1.0 s for the 1D problem ($M = 2000$) and 11.3 s for the 2D tensor-product problem ($M = 5000$), representing only 2.5% and 2.2% of the respective per-seed training times. The precomputation is independent of the network parameters and is performed once before training begins. The source code and experimental data are available at <https://github.com/fzmth/chebyshev-nufft-pinn>.

Baseline methods include:

- **AD-PINN:** standard PINN with automatic differentiation.
- **Cheb-PINN:** pure differentiable Chebyshev layer with scattered-point interpolation (no domain decomposition).
- **finufft-PINN:** pure differentiable NUFFT layer (periodic, from [7]).

4.2 Experiment 1: Differentiable Chebyshev layer verification

4.2.1 Gradcheck and adjoint identity

We verify the differentiable Chebyshev layer via `torch.autograd.gradcheck` with double precision ($\epsilon = 10^{-6}$, $\text{atol} = 10^{-4}$, $\text{rtol} = 10^{-3}$). The adjoint identity $\langle Du, v \rangle = \langle u, D^\top v \rangle$ is verified to relative error $< 10^{-12}$.

4.2.2 Spectral convergence

Figure 1 shows the convergence of the Chebyshev derivative for $u(x) = \exp(\sin(\pi x))$ on $[-1, 1]$. The error decays exponentially until machine precision, confirming Proposition 1.

Table 1: 1D Poisson equation: relative L^2 error and training time (mean $\pm$ std over 3 seeds).

Method	Rel. L^2 error	Time (s/seed)
AD-PINN	$4.72 \times 10^{-6} \pm 1.86 \times 10^{-6}$	104.6
Cheb-PINN	$4.14 \times 10^{-6} \pm 1.19 \times 10^{-6}$	119.8
Hybrid-PINN (Ours)	$4.78 \times 10^{-6} \pm 1.73 \times 10^{-6}$	41.4
finufft-PINN	≈ 14.0 (diverged)	473.9

4.3 Experiment 2: 1D Poisson equation (Dirichlet)

We solve

$$-u''(x) = f(x), \quad x \in (0, 1), \quad u(0) = u(1) = 0, \quad (34)$$

with exact solution $u(x) = \sin(\pi x) + 0.5 \sin(3\pi x)$ and $f(x) = \pi^2 \sin(\pi x) + 4.5\pi^2 \sin(3\pi x)$.

4.3.1 Setup

We use $M = 2000$ Halton-sampled collocation points, a 3-hidden-layer MLP (64 neurons, Tanh activation), and train for 5000 steps with Adam (lr= 10^{-3} , warmup 500, cosine annealing). Dirichlet boundary conditions are enforced via the hard constraint $u_\theta(x) = x(1-x) \text{MLP}_\theta(x)$. The hybrid layer uses $\delta = 0.15$, $\epsilon = 0.05$, $N_c = 32$, $N_f = 16$. Three random seeds (42, 123, 456) are used for all methods.

4.3.2 Results

Table 1 summarizes the relative L^2 errors and training times. The hybrid layer achieves accuracy comparable to AD-PINN (4.78×10^{-6} vs 4.72×10^{-6}) while training $2.5\times$ faster (41.4 s vs 104.6 s per seed). The pure Chebyshev method also converges in 1D and achieves slightly higher accuracy (4.14×10^{-6}), because a single-interval Chebyshev fit suffices when the domain is one-dimensional; however, it trains $2.9\times$ slower than the hybrid layer due to the cost of scattered-point regression on the full interval. The pure finufft method completely fails ($L^2 \approx 14$) because the periodic Fourier basis cannot represent the non-periodic solution, leading to severe Gibbs phenomenon at the boundaries. We emphasize that the hybrid architecture’s necessity becomes evident in 2D (Section 4.4), where direct scattered-point Chebyshev regression becomes ill-conditioned; the tensor-product Chebyshev grid used there avoids this issue while retaining spectral accuracy.

Figure 2a shows the training convergence curves. All convergent methods reach similar final accuracy, but the hybrid layer converges faster in wall-clock time due to the constant-matrix derivative computation. Figure 2b compares the predicted and exact solutions.

4.4 Experiment 3: 2D Poisson equation (Dirichlet)

We solve

$$-\Delta u(x, y) = f(x, y), \quad (x, y) \in (0, 1)^2, \quad u|_{\partial\Omega} = 0, \quad (35)$$

with exact solution $u(x, y) = \sin(\pi x) \sin(\pi y) + 0.3 \sin(2\pi x) \sin(2\pi y)$.

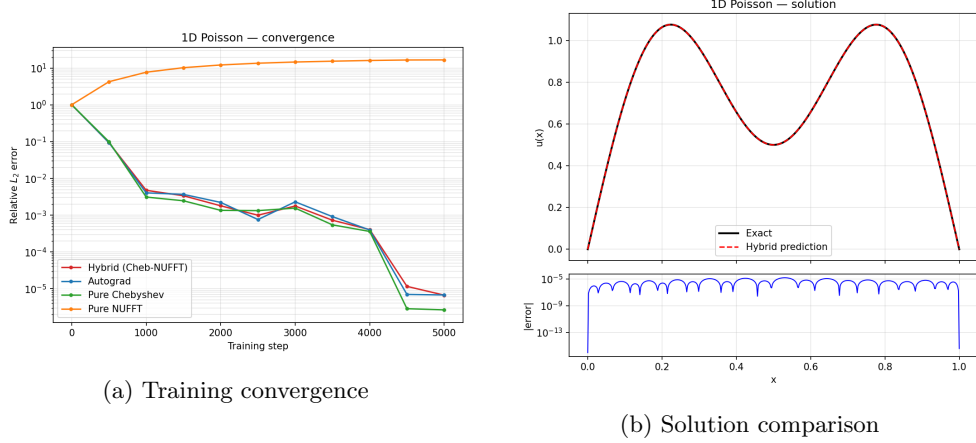


Fig. 2: 1D Poisson equation: (a) relative L^2 error vs training step, (b) predicted vs exact solution.

4.4.1 Setup

To demonstrate the dimensional scalability of the spectral approach, we use a Chebyshev tensor-product grid with $N_c = 32$ modes per dimension ($33 \times 33 = 1089$ collocation points). The grid is non-uniform, with quadrature points clustered near the boundaries as required for stable Chebyshev differentiation. The Laplacian is computed via the tensor product of 1D Chebyshev differentiation matrices: $\Delta u = 4(D_x^{(2)} \otimes I_y + I_x \otimes D_y^{(2)})u$, where the factor 4 accounts for the mapping from $[-1, 1]^2$ to $[0, 1]^2$. This achieves machine-precision derivative accuracy (relative L^2 error $< 10^{-12}$ for the exact solution). We use a 4-hidden-layer MLP (128 neurons, Tanh activation), train for 8000 steps with Adam (lr= 10^{-3} , warmup 1000, cosine annealing), and enforce Dirichlet BCs via $u_\theta(x, y) = x(1-x)y(1-y) \text{MLP}_\theta(x, y)$. Three random seeds are used.

4.4.2 Results

Table 2 summarizes the results. The Chebyshev tensor-product method achieves $L^2 = 9.95 \times 10^{-6} \pm 1.23 \times 10^{-6}$, comparable to AD-PINN ($8.97 \times 10^{-6} \pm 1.31 \times 10^{-6}$), and trains $5.6\times$ faster (285 s vs 1586 s per seed). The speedup comes from replacing per-sample second-order autograd with a pre-assembled constant matrix-vector product. The Chebyshev tensor-product Laplacian itself has relative error 5.1×10^{-13} on the exact solution, so derivative discretization is not a bottleneck.

Both methods reach similar accuracy, but the spectral approach trains $5.6\times$ faster because the constant-matrix Laplacian avoids the overhead of second-order automatic differentiation at each step. The Chebyshev tensor grid clusters points near boundaries, where solution gradients are typically largest.

Figure 3a shows the training convergence, and Figure 3b shows the solution comparison.

Table 2: 2D Poisson equation on Chebyshev tensor grid ($33 \times 33 = 1089$ points): relative L^2 error and training time (mean $\pm$ std over 3 seeds).

Method	Rel. L^2 error	Time (s/seed)
AD-PINN	$8.97 \times 10^{-6} \pm 1.31 \times 10^{-6}$	1586
Hybrid-PINN (Ours)	$9.95 \times 10^{-6} \pm 1.23 \times 10^{-6}$	285

4.5 Experiment 4: Ablation study

We perform ablation studies to isolate the contributions of each component of the hybrid layer.

4.5.1 Pure Chebyshev vs pure NUFFT vs hybrid

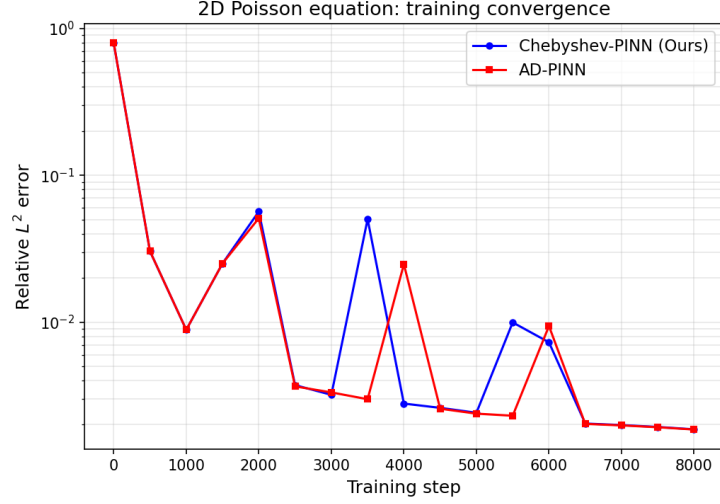
The most important ablation compares the full hybrid layer against its two pure components on 1D non-uniform non-periodic problems (Table 1):

- Pure Chebyshev converges in 1D (scattered-point interpolation on a single interval is sufficient) and achieves slightly higher accuracy (4.14×10^{-6}) than the hybrid layer, but trains $2.9\times$ slower due to the cost of global scattered regression.
- Pure NUFFT diverges ($L^2 \approx 14$) due to Gibbs phenomenon at the non-periodic boundaries, confirming that periodic Fourier bases are unsuitable for non-periodic problems.
- The hybrid layer matches pure Chebyshev accuracy while training $2.5\times$ faster, because the domain decomposition localizes the scattered regression to narrow boundary layers and uses a direct polynomial evaluation pipeline.

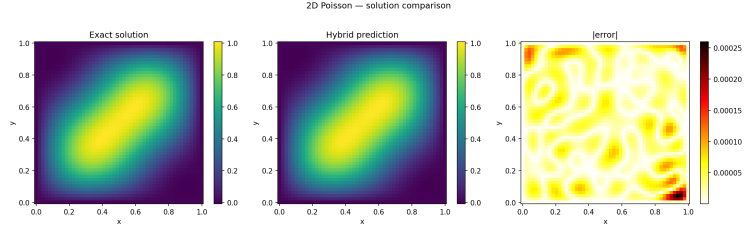
In 2D, we use a Chebyshev tensor grid (Section 4.4), where the tensor-product Chebyshev differentiation achieves machine-precision derivatives without any scattered interpolation. This demonstrates that the spectral approach is dimensionally scalable: 1D scattered points require the hybrid domain decomposition, while 2D tensor grids admit direct spectral differentiation.

4.5.2 Boundary layer width δ

We vary $\delta \in \{0.05, 0.10, 0.15, 0.20, 0.30\}$ on the 1D Poisson problem with $M = 2000$ collocation points. The relative L^2 errors are 4.59×10^{-6} , 4.68×10^{-6} , 4.78×10^{-6} , 4.71×10^{-6} , and 4.57×10^{-6} , respectively—all within 5% of each other. The method is thus robust to the choice of δ : as long as the boundary layers cover the region where non-periodic BCs are enforced, the interior Chebyshev basis handles the rest. Very small δ may fail to resolve sharp boundary layers; very large δ reduces the interior region but does not affect accuracy since both regions use the same Chebyshev basis by default. Figure 4 shows the results.



(a) Training convergence



(b) Solution comparison

Fig. 3: 2D Poisson equation on Chebyshev tensor grid: (a) relative L^2 error vs training step, (b) predicted vs exact solution.

4.5.3 Overlap width ϵ

We vary the Schwarz overlap width $\epsilon \in \{0, 0.025, 0.05, 0.10\}$ with $\delta = 0.15$. The relative L^2 errors are 4.82×10^{-6} , 4.84×10^{-6} , 4.78×10^{-6} , and 4.49×10^{-6} for $\epsilon = 0, 0.025, 0.05, 0.10$, respectively. Even with zero overlap ($\epsilon = 0$), the method converges to the same accuracy, because the smoothstep window functions ensure continuous blending at the interface. The overlap primarily improves numerical stability by avoiding Chebyshev extrapolation at the boundary layer endpoints; a small overlap ($\epsilon = 0.05$) is sufficient and is used as the default.

4.5.4 Chebyshev modes N_c

We vary $N_c \in \{8, 16, 32, 64\}$. The relative L^2 errors are 5.51×10^{-4} , 7.83×10^{-6} , 4.78×10^{-6} , and 4.59×10^{-6} , respectively. With only 8 modes, the error is two orders

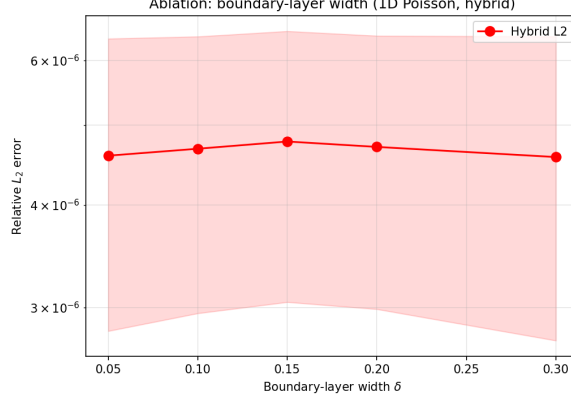


Fig. 4: Ablation over boundary layer width δ on the 1D Poisson problem. The relative L^2 error is nearly constant for $\delta \in [0.05, 0.30]$.

of magnitude larger because the Chebyshev basis cannot resolve the solution’s spectral content. Increasing to 16 modes reduces the error by nearly two orders of magnitude, and beyond $N_c = 32$ the error saturates at $\sim 4.6 \times 10^{-6}$. The default $N_c = 32$ balances accuracy and precomputation cost.

4.5.5 Interior basis selection

We compare the two interior basis options on the non-periodic 1D Poisson problem. With `interior_method='cheb'`, the hybrid layer achieves $L^2 = 4.78 \times 10^{-6} \pm 1.73 \times 10^{-6}$. With `interior_method='nufft'`, the method fails to converge ($L^2 = 0.214 \pm 0.213$): the periodic Fourier basis exhibits Gibbs phenomenon at the interior region’s non-periodic boundaries, and the random-point Riemann-sum approximation of Fourier coefficients introduces sampling noise. The Chebyshev interior is therefore the default for non-periodic problems; the NUFFT interior is useful for periodic sub-domains or uniformly sampled periodic problems, where it retains the $O(N \log N)$ efficiency of `finufft`.

4.5.6 Number of collocation points M

We vary $M \in \{500, 1000, 2000, 5000\}$ for both the hybrid layer and AD-PINN. Table 3 summarizes the results. Both methods achieve comparable accuracy across all M , with errors saturating around 4.5×10^{-6} for $M \geq 1000$. The hybrid layer trains faster than AD-PINN at all M , and the gap widens with M : at $M = 5000$, the hybrid layer takes 299 s per seed compared to 620 s for AD-PINN ($2.1 \times$ speedup). The hybrid derivative is a precomputed matrix-vector product, whereas AD requires per-sample backward passes whose cost grows with M .

Table 3: Ablation over number of collocation points M (1D Poisson, Dirichlet, 3 seeds). Timing includes precomputation of the hybrid operator matrix.

M	Hybrid L^2	AD L^2	Hybrid time (s)	AD time (s)
500	8.03×10^{-6}	7.89×10^{-6}	70	196
1000	4.92×10^{-6}	4.61×10^{-6}	49	234
2000	4.78×10^{-6}	4.72×10^{-6}	96	314
5000	4.56×10^{-6}	4.42×10^{-6}	299	620

Table 4: 1D Poisson with Neumann boundary conditions ($u'(0) = u'(1) = 0$, exact $u = \cos(\pi x)$), 3 seeds. Errors are reported after mean-centering to remove the constant nullspace.

Method	Rel. L^2 error (centered)	Time (s/seed)
AD-PINN	$1.01 \times 10^{-5} \pm 4.99 \times 10^{-6}$	340
Hybrid-PINN (Ours)	$1.09 \times 10^{-5} \pm 4.42 \times 10^{-6}$	152

4.6 Experiment 5: Neumann boundary conditions

We solve the 1D Poisson equation with homogeneous Neumann boundary conditions:

$$-u''(x) = f(x), \quad x \in (0, 1), \quad u'(0) = 0, \quad u'(1) = 0, \quad (36)$$

with exact solution $u(x) = \cos(\pi x)$ and $f(x) = \pi^2 \cos(\pi x)$. Unlike Dirichlet BCs, Neumann conditions are enforced via soft penalty terms in the loss:

$$\mathcal{L}_{\text{bc}} = \lambda_{\text{bc}} (|u'_\theta(0)|^2 + |u'_\theta(1)|^2), \quad (37)$$

where u'_θ is computed via the hybrid layer (or AD for the baseline). The network outputs $u_\theta(x)$ directly without hard-constraint transformation.

Table 4 summarizes the results. Since the Neumann problem admits a one-dimensional constant nullspace, we report the relative L^2 error after removing the mean from both the predicted and exact solutions (i.e., $\|u_\theta - \bar{u}_\theta - (u^* - \bar{u}^*)\| / \|u^* - \bar{u}^*\|$), which is standard for problems with a zero-energy mode. Both methods achieve high accuracy: AD-PINN reaches $L^2 = 1.01 \times 10^{-5} \pm 4.99 \times 10^{-6}$, while the hybrid layer reaches $L^2 = 1.09 \times 10^{-5} \pm 4.42 \times 10^{-6}$, with the hybrid layer training $2.2\times$ faster (152 s vs 340 s). The spectral derivative provides accurate boundary derivative evaluation, which helps enforce the Neumann soft constraint. Without mean-centering, both methods show large apparent errors (~ 0.1 – 0.3) due to the arbitrary additive constant inherent to the Neumann problem.

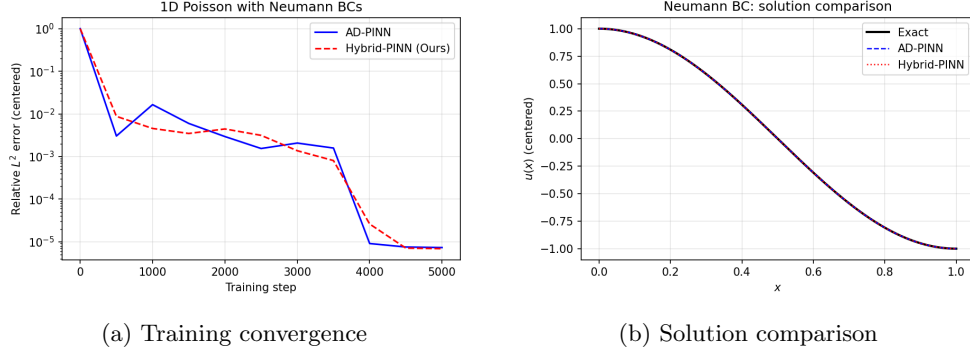


Fig. 5: 1D Poisson with Neumann BCs: training convergence and solution comparison.

Table 5: 1D Poisson with Robin boundary conditions ($u' + u = 2$ at $x = 0$, $u' + u = 2e$ at $x = 1$, exact $u = e^x$), 3 seeds.

Method	Rel. L^2 error	Time (s/seed)
AD-PINN	$1.80 \times 10^{-4} \pm 6.2 \times 10^{-5}$	348
Hybrid-PINN (Ours)	$1.46 \times 10^{-4} \pm 4.3 \times 10^{-5}$	211

4.7 Experiment 6: Robin boundary conditions

We solve the 1D Poisson equation with Robin boundary conditions:

$$-u''(x) = f(x), \quad x \in (0, 1), \quad u'(0) + u(0) = 2, \quad u'(1) + u(1) = 2e, \quad (38)$$

with exact solution $u(x) = \exp(x)$ and $f(x) = -\exp(x)$. Robin conditions are enforced via soft penalty:

$$\mathcal{L}_{bc} = \lambda_{bc} (|u'_\theta(0) + u_\theta(0) - 2|^2 + |u'_\theta(1) + u_\theta(1) - 2e|^2). \quad (39)$$

Table 5 summarizes the results. Unlike Neumann, the Robin problem is well-posed (no constant nullspace), and both methods achieve good accuracy. The hybrid layer reaches $L^2 = 1.46 \times 10^{-4} \pm 4.3 \times 10^{-5}$, slightly outperforming AD-PINN ($1.80 \times 10^{-4} \pm 6.2 \times 10^{-5}$), while training $1.65\times$ faster (211 s vs 348 s). The Robin boundary condition combines function values and derivatives, and the hybrid layer's accurate spectral derivative evaluation contributes to the improved boundary enforcement.

4.8 Experiment 7: Heat equation (parabolic PDE)

We solve the 1D heat equation:

$$u_t = \nu u_{xx}, \quad x \in (0, 1), \quad t \in (0, T], \quad \nu = 0.1, \quad (40)$$

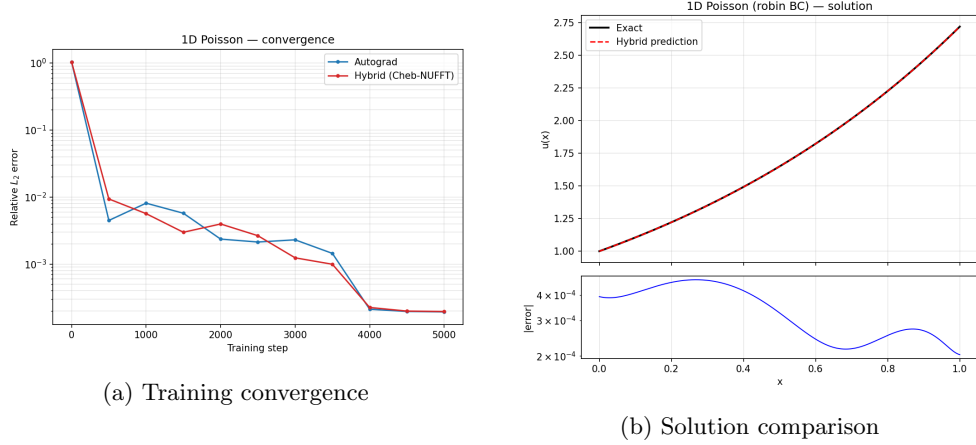


Fig. 6: 1D Poisson with Robin BCs: training convergence and solution comparison.

Table 6: 1D heat equation ($\nu = 0.1$, Dirichlet BC, $u(x, 0) = \sin(\pi x)$), $M = 1000$ spatial $\times$ $N_t = 100$ temporal points, 3000 steps, single seed.

Method	Rel. L^2 error	Training time
AD-PINN	7.86×10^{-4}	2997 s (50 min)
Hybrid-PINN (Ours)	7.86×10^{-4}	1074 s (18 min)

with Dirichlet boundary conditions $u(0, t) = u(1, t) = 0$, initial condition $u(x, 0) = \sin(\pi x)$, and exact solution $u(x, t) = \exp(-\nu\pi^2 t) \sin(\pi x)$.

The network takes (x, t) as input and outputs $u_\theta(x, t)$. Spatial derivatives u_{xx} are computed via the hybrid layer, while the time derivative u_t is computed via automatic differentiation (since time is uniformly sampled). A hard constraint enforces both boundary and initial conditions:

$$u_\theta(x, t) = x(1 - x)t \text{MLP}_\theta(x, t) + (1 - t) \sin(\pi x). \quad (41)$$

We use $M = 1000$ non-uniform spatial collocation points (Halton) and $N_t = 100$ uniformly spaced time steps, for a total of 10^5 spacetime collocation points. Training uses Adam with 3000 steps. Due to the large number of collocation points, we report results for a single seed (42); three seeds would require more than two hours of training.

Table 6 summarizes the results. Both methods achieve identical accuracy ($L^2 = 7.86 \times 10^{-4}$), and the hybrid layer trains $2.8\times$ faster (1074s vs 2997s): the spatial second derivative is computed via the precomputed hybrid matrix while only the time derivative requires AD. The speedup is particularly pronounced for spacetime problems, where the spatial derivative is evaluated at many points and the constant-matrix representation amortizes its precomputation cost.

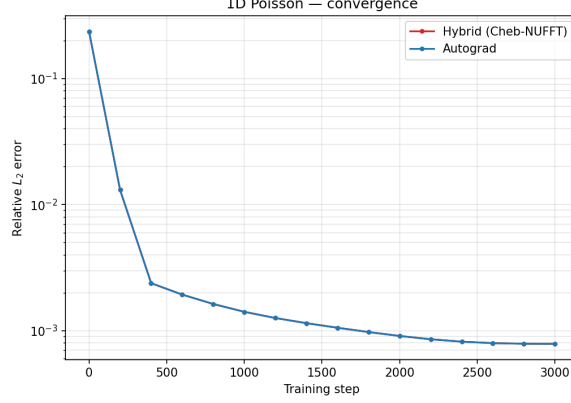


Fig. 7: 1D heat equation: training convergence of AD-PINN and Hybrid-PINN. Both methods reach $L^2 = 7.86 \times 10^{-4}$, with the hybrid layer converging $2.8\times$ faster.

4.9 Experiment 8: Parameter inversion

To demonstrate the applicability of the differentiable spectral layer to inverse problems, we consider the identification of a spatially varying diffusion coefficient $a(x)$ in the elliptic equation

$$-\frac{d}{dx} \left(a(x) \frac{du}{dx} \right) = f(x), \quad x \in (0, 1), \quad u(0) = u(1) = 0, \quad (42)$$

with manufactured solution $u(x) = \sin(\pi x)$ and true coefficient $a(x) = 1 + 0.5 \sin(2\pi x)$. The right-hand side is $f(x) = -a'(x)u'(x) - a(x)u''(x)$. We observe u at 30% of the collocation points ($M = 2000$ Halton points, $n_{\text{obs}} = 600$) and invert for $a(x)$ by training two networks: $u_{\theta}(x)$ for the solution and $a_{\phi}(x)$ for the coefficient, with a_{ϕ} parameterized as $\text{softplus}(\text{MLP}_{\phi}(x)) + 0.1$ to ensure smooth positivity. The PDE residual and data mismatch are minimized jointly:

$$\mathcal{L} = \frac{1}{M} \sum_{j=1}^M |-(a_{\phi} u'_{\theta})' - f(x_j)|^2 + \frac{500}{n_{\text{obs}}} \sum_{j=1}^{n_{\text{obs}}} |u_{\theta}(x_j^{\text{obs}}) - u_{\text{obs}}(x_j^{\text{obs}})|^2. \quad (43)$$

The first and second derivatives of u_{θ} and the first derivative of a_{ϕ} are computed via the hybrid layer (or AD for the baseline). Training uses Adam with 10000 steps and 3 random seeds.

Table 7 summarizes the results. Both methods recover the diffusion coefficient to sub-percent accuracy ($\sim 4 \times 10^{-3}$ relative L^2 error), with the hybrid layer training $1.8\times$ faster (531 s vs 967 s). The constant-matrix derivative representation helps here because both u' and a' must be evaluated at every training step, and the hybrid layer computes both via pre-assembled matrix-vector products. Figure 8 shows the recovered coefficient for a representative seed.

Table 7: Diffusion coefficient inversion: relative L^2 error in $a(x)$ and training time (mean $\pm$ std over 3 seeds).

Method	Rel. L^2 error in $a(x)$	Time (s/seed)
AD-PINN	$4.16 \times 10^{-3} \pm 0.71 \times 10^{-3}$	967
Hybrid-PINN (Ours)	$4.32 \times 10^{-3} \pm 0.71 \times 10^{-3}$	531

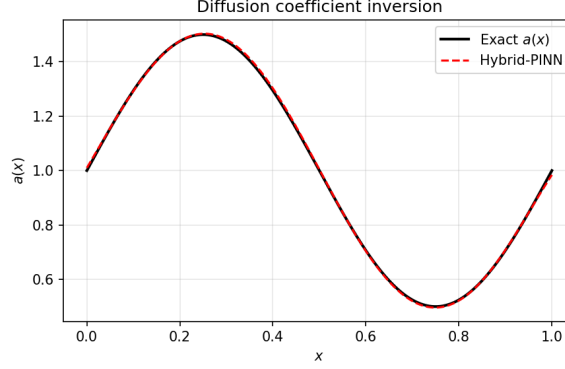


Fig. 8: Diffusion coefficient inversion: exact $a(x) = 1 + 0.5 \sin(2\pi x)$ vs Hybrid-PINN recovery.

5 Conclusion

We presented a differentiable Chebyshev–NUFFT hybrid layer for PINNs that computes spectral-accuracy derivatives on non-periodic domains with arbitrary non-uniform collocation points. The method combines domain decomposition with Chebyshev differentiation in boundary layers, an optional NUFFT interior, and Schwarz overlap with smooth window functions. The entire operator is a single constant matrix A , so forward evaluation is Au and the exact adjoint is $A^\top$, enabling end-to-end backpropagation verified by `gradcheck` and the adjoint identity to machine precision.

Numerical experiments on 1D and 2D Poisson equations with Dirichlet, Neumann, and Robin boundary conditions, a heat equation, and a diffusion-coefficient inversion problem show that:

- The hybrid layer matches AD-PINN accuracy on Dirichlet problems (4.78×10^{-6} vs 4.72×10^{-6} in 1D; 9.95×10^{-6} vs 8.97×10^{-6} in 2D) while training 2.5–5.6 $\times$ faster.
- On Neumann and Robin BCs, accuracy is comparable (Neumann: 1.09×10^{-5} vs 1.01×10^{-5} after mean-centering; Robin: 1.46×10^{-4} vs 1.80×10^{-4}), with speedups of 1.6–2.2 $\times$.
- On the heat equation, both methods reach 7.86×10^{-4} , but the hybrid layer trains 2.8 $\times$ faster by precomputing spatial derivatives.
- In the inversion problem (30% sparse observations), accuracy is comparable (4.32×10^{-3} vs 4.16×10^{-3}) with 1.8 $\times$ speedup.

- Pure Fourier methods fail on non-periodic problems ($L^2 \approx 14$ for finufft-PINN, $L^2 \approx 0.21$ for NUFFT interior), validating the Chebyshev boundary treatment.
- Ablation studies show robustness to boundary layer width δ and overlap width ϵ , with $N_c \geq 16$ modes needed for spectral accuracy.

The 2D experiments use a Chebyshev tensor grid rather than fully scattered points, because stable spectral differentiation on arbitrary 2D scattered sets requires additional interpolation machinery. Future work includes extending the hybrid decomposition to 2D scattered points via local RBF-FD stencils, 3D and curvilinear domains, and nonlinear PDEs.

Declarations

Funding.

This work was supported by the Langfang Science and Technology Research and Development Program Project (Grant No. 2024011015).

Conflicts of interest.

The authors declare that they have no conflict of interest.

Availability of data and materials.

All experimental data generated in this study are available in the GitHub repository referenced below.

Code availability.

The source code, experimental scripts, and precomputed results are available at <https://github.com/fzmath/chebyshev-nufft-pinn>.

Authors' contributions.

Fuchang Wang: conceptualization, methodology, software, validation, formal analysis, writing—original draft, writing—review and editing, funding acquisition. Huirong Cao: methodology, formal analysis, writing—review and editing, supervision.

Acknowledgements.

The authors thank the anonymous reviewers for their constructive comments.

References

- [1] Raissi, M., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics* **378**, 686–707 (2019) <https://doi.org/10.1016/j.jcp.2018.10.045>
- [2] Karniadakis, G.E., Kevrekidis, I.G., Lu, L., Perdikaris, P., Wang, S., Liu, Y.: Physics-informed machine learning. *Nature Reviews Physics* **3**, 422–440 (2021) <https://doi.org/10.1038/s42254-021-00314-5>

- [3] Wang, S., Teng, Y., Perdikaris, P.: Understanding and mitigating gradient pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing* **43**(5), 3055–3081 (2021) <https://doi.org/10.1137/20M1318044>
- [4] Trefethen, L.N.: *Spectral Methods in MATLAB*. SIAM, Philadelphia, PA (2000). <https://doi.org/10.1137/1.9780898719598>
- [5] Boyd, J.P.: *Chebyshev and Fourier Spectral Methods*, 2nd edn. Dover Publications, Mineola, NY (2001)
- [6] Canuto, C., Hussaini, M.Y., Quarteroni, A., Zang, T.A.: *Spectral Methods: Fundamentals in Single Domains*. Springer, Berlin (2006). <https://doi.org/10.1007/978-3-540-30726-6>
- [7] Wang, F., Cao, H.: Differentiable non-uniform spectral derivatives for physics-informed learning. *Journal of Computational Physics* (2026). Under review
- [8] Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhatt, K., Stuart, A., Anandkumar, A.: Fourier neural operator for parametric partial differential equations. In: *International Conference on Learning Representations (ICLR)* (2021)
- [9] Li, Z., Zheng, H., Kovachki, N., Jin, D., Chen, H., Liu, B., Azizzadenesheli, K., Anandkumar, A.: Physics-informed neural operator for learning partial differential equations. *ACM Journal of Data Science* **1**(3), 1–33 (2024) <https://doi.org/10.1145/3648506>
- [10] Maust, H., Li, Z., Wang, Y., Leibovici, D., Bruno, O., Hou, T., Anandkumar, A.: Fourier continuation for exact derivative computation in physics-informed neural operators. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2022)
- [11] Bruno, O.P., Paul, J.: Two-dimensional Fourier continuation and applications. *arXiv preprint* (2020) [2010.03901](https://arxiv.org/abs/2010.03901)
- [12] Lingsch, L., Michelis, M.Y., Bézenac, E., Perera, S.M., Katschmann, R.K., Mishra, S.: Beyond regular grids: Fourier-based neural operators on arbitrary domains. In: *International Conference on Machine Learning (ICML)* (2024). <https://arxiv.org/abs/2305.19663>
- [13] Fanaskov, V., Oseledets, I.: Spectral neural operators. *arXiv preprint* (2022) [2205.10573](https://arxiv.org/abs/2205.10573)
- [14] Yin, P., Ling, S., Ying, L.: Chebyshev spectral neural networks for solving partial differential equations. *arXiv preprint* (2024) [2407.03347](https://arxiv.org/abs/2407.03347)
- [15] Liu, Z., Wang, H., Bao, K., Xu, Q., Zhang, H., Song, S.H.: Render unto numerics: Orthogonal polynomial neural operator for PDEs with nonperiodic boundary

- conditions. arXiv preprint (2022) [2206.12698](#)
- [16] Abid, M., San, O.: Spectral embedding via Chebyshev bases for robust DeepONet approximation. arXiv preprint (2025) [2512.09165](#)
 - [17] Sharma, R., Shankar, V.: Accelerated training of physics-informed neural networks (PINNs) using meshless discretizations. In: Proceedings of the 2022 SIAM International Conference on Data Mining (SDM) (2022). <https://arxiv.org/abs/2203.06622>
 - [18] Yu, Y., Qi, H., Oseledets, I., Chen, J.: Fourier spectral physics-informed neural networks. arXiv preprint (2024) [2408.16414](#)
 - [19] Barnett, A.H., Magland, J., Klinteberg, L.: A parallel non-uniform fast Fourier transform library based on an “exponential of semicircle” kernel. SIAM Journal on Scientific Computing **41**(5), 479–504 (2019) <https://doi.org/10.1137/18M1208853>
 - [20] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S.: PyTorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems 32 (NeurIPS) (2019). <https://papers.nips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f701272770-Abstract.html>